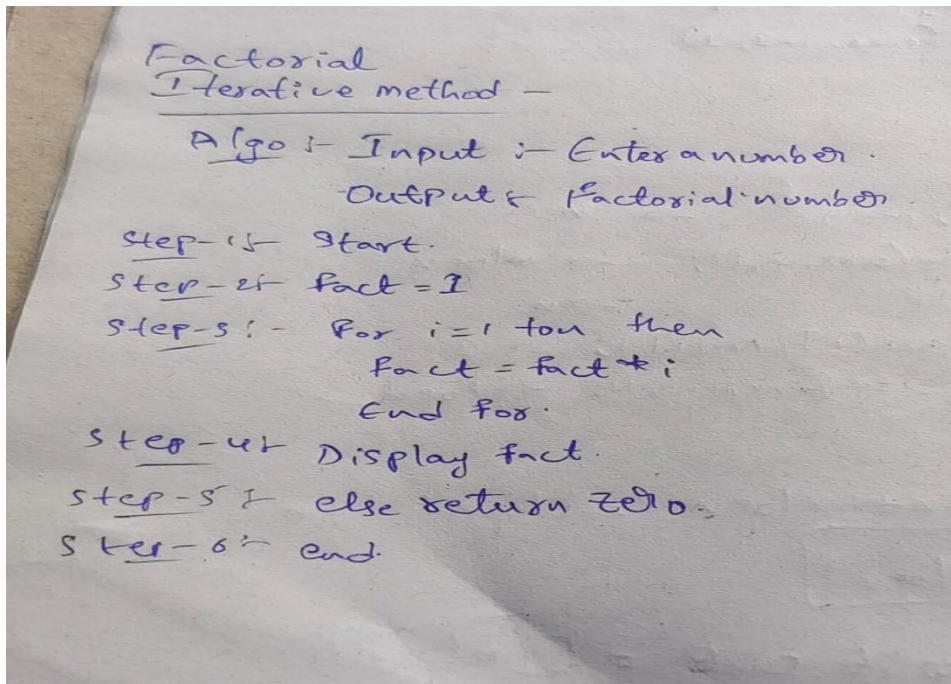


PRACTICAL-04

AIM: Implementation and Time analysis of factorial program using iterative and recursive method

1.) ITERATIVE METHOD

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

int main(){

    int n;

    int fact = 1;

    cout << "enter a number:";

    cin >> n;

    for(int i = 1; i <= n; i++){

        fact = fact * i;

    }

    cout << "Factorial of " << n << " = " << fact;
```

```
return 0;
```

```
}
```

OUTPUT:

```
Output
enter a number:5
Factorial of 5 = 120

=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time Complexity:-

fact = 1
for (i = 1 to n)
fact = fact * i
 $T(n) = 1 + n + n = 2n + 1$
 $T(n) = O(n)$

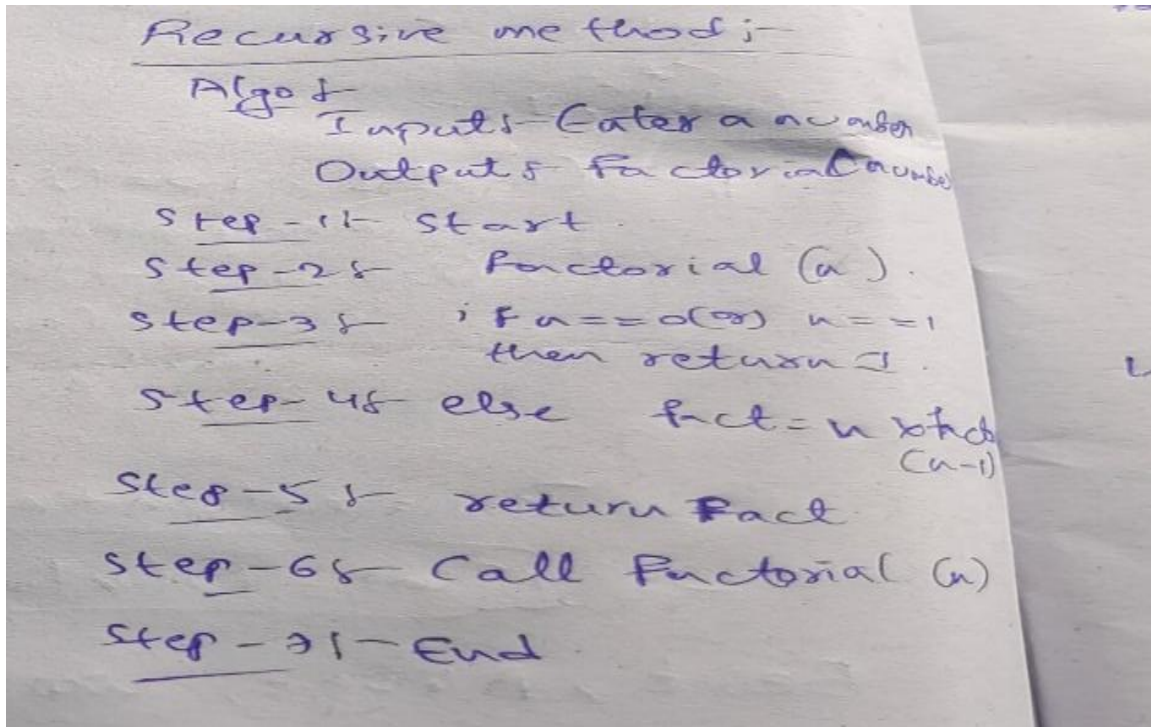
Average case -
loop always executes n times
i = 1 → 1
i = 2 → 1
i = 3 → 1
i = n → 1
no. of iterations = n
 $T(n) = O(n)$

Best case
loop runs from 1 to n
fact = 1 → 1
for i = 1 to n → n
fact = fact * i → n
 $T(n) = 1 + n + n$
 $\Rightarrow T(n) = O(n)$

worst case -
It is also, loop executes n times.
n → n/2 → n/4 → ...
1, 2, 3, ... n
 $T(n) = 1 + n + n = 2n + 1$
 $T(n) = O(n)$

2.) RECURSIVE METHOD:

ALGORITHM:



Recursive method:-

Algo:-
Inputs:- Enter a number
Outputs:- Factorial number

Step-1:- Start

Step-2:- Factorial (n).

Step-3:- if $n == 0$ or $n == 1$ then return 1.

Step-4:- else $fact = n \times fact(n-1)$

Step-5:- return fact

Step-6:- Call Factorial (n)

Step-7:- End

PROGRAM:

```
#include <iostream>

using namespace std;

class factorial{
public:
    int factr(int n){
        if(n == 0 || n == 1){
            return 1;
        }
        else{
            int result = n*factr(n-1);
            return result;
        }
    }
}
```

```
};

int main(){

    int n,result;

    factorial obj;

    cout << "enter a number:";

    cin>>n;

    result = obj.factr(n);

    cout << "factorial of " << n << " is " << result;

    return 0;

}
```

OUTPUT:

```
Output

enter a number:7
factorial of 7 is 5040

=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time Complexity

Factorial (n) $\rightarrow n$

if $(n == 0 \text{ || } n == 1) \rightarrow n$

$n * \text{factorial}(n-1) \rightarrow (n-1)$

return fact. $\rightarrow n$

$T(n) = n + n + (n-1) + n$

$= 4n - 1$

$T(n) = O(n)$

Best case :-

When $n = 0$ (or) $n = 1$

Factorial (1) $\rightarrow 1$

$T(n) = O(1)$

Average case :-

No. of calls is 5

ex: $n = 5 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$

$T(n) = O(n)$

Worst case :-

large number of n. continued. until it reaches 1.

ex: $n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots \rightarrow 1$

$T(n) = O(n)$

CONCLUSION:

Both iterative and recursive methods are used to find factorial. Both take $O(n)$ time, but the iterative method uses less memory and is simpler to implement.